

Лабораторная работа №8

Композиция и агрегация классов. Статические члены

Цель работы:

1. Научиться проектировать отношения между классами: композицию, агрегацию и ассоциацию
2. Освоить реализацию композиции (поля-объекты, вложение структур) и агрегации (указатели, ссылки)
3. Понять разницу между композицией и агрегацией с точки зрения времени жизни объектов
4. Научиться использовать статические поля и методы для подсчёта объектов и реализации общих данных для всех экземпляров класса
5. Получить навыки правильного управления памятью при разных типах отношений между классами

Теоретическое обоснование

1. Отношения между классами

В объектно-ориентированном программировании классы редко существуют изолированно. Они взаимодействуют, объединяются в структуры, используют друг друга. Существует три основных типа отношений:

Отношение	Характер связи	Пример
Ассоциация	«использует», «знает о»	Врач → Пациент
Агрегация	«состоит из» (часть может жить без целого)	Автомобиль → Колёса
Композиция	«состоит из» (часть не может жить без целого)	Дом → Комнаты

2. Композиция

Композиция - это отношение «часть–целое», при котором «части» не могут существовать без «целого». Целое полностью контролирует время жизни частей.

Ключевая характеристика: Часть умирает вместе с целым.

Реализация композиции в C++

```
// Композиция через поле-объект (часть создаётся внутри целого)
class Room {
```

```

private:
    double area;
    string purpose;
public:
    Room(double a, const string& p) : area(a), purpose(p) {}
};

class House {
private:
    string address;
    Room livingRoom; // композиция! комнаты создаются внутри дома
    Room kitchen;
public:
    House(const string& addr, double area1, double area2)
        : address(addr), livingRoom(area1, "Гостиная"), kitchen(area2,
"Кухня") {}
};

// Композиция через динамическое выделение памяти
class Engine {
public:
    Engine(int hp) { cout << "Двигатель создан\n"; }
    ~Engine() { cout << "Двигатель уничтожен\n"; }
};

class Car {
private:
    Engine* engine; // указатель, но управляет памятью сам
public:
    Car(int hp) : engine(new Engine(hp)) {} // создаёт двигатель
    ~Car() { delete engine; } // уничтожает двигатель
};

// Композиция через умные указатели (рекомендуемый способ)
#include <memory>

class Car {
private:
    std::unique_ptr<Engine> engine; // автоматическое управление
памятью
public:
    Car(int hp) : engine(std::make_unique<Engine>(hp)) {}
};

```

```
    // деструктор не нужен - unique_ptr сам очистит память  
};
```

3. Агрегация

Агрегация - это отношение «часть—целое», при котором «части» могут существовать независимо от «целого» и могут принадлежать разным «целым» одновременно.

Ключевая характеристика: Часть может существовать без целого.

Реализация агрегации в C++

```
class Book {  
private:  
    string title;  
public:  
    Book(const string& t) : title(t) {}  
    string getTitle() const { return title; }  
};
```

```
class Library {  
private:  
    string name;  
    vector<Book*> books; // агрегация: книги существуют независимо  
public:  
    Library(const string& n) : name(n) {}  
  
    void addBook(Book* book) {  
        books.push_back(book);  
    }  
    // В деструкторе книги НЕ удаляются!  
};
```

```
int main() {  
    Book b1("Война и мир");    // книга создаётся независимо  
    Book b2("Преступление и наказание");  
  
    Library lib("Городская"); // библиотека создаётся  
    lib.addBook(&b1);          // книга добавляется в библиотеку  
    lib.addBook(&b2);  
  
    // Библиотека уничтожается, но книги остаются!  
    return 0;  
}
```

4. Сравнение композиции и агрегации

Характеристика	Ассоциация	Агрегация	Композиция
Сила связи	Слабая	Средняя	Сильная
Время жизни частей	Независимое	Дольше или равное целому	Вместе с целым
Целое отвечает за уничтожение	Нет	Нет	Да
Целое отвечает за создание	Нет	Нет	Да
Каскадное удаление	Нет	Нет	Да
Реализация	Указатели, ссылки	Указатели	Поля-объекты, unique_ptr

5. Статические члены класса

Статическое поле - принадлежит не отдельному объекту, а всему классу в целом. Существует в единственном экземпляре для всех объектов класса.

Статический метод - может быть вызван без создания объекта класса.

```
class Student {  
private:  
    string name;  
    static int totalCount; // объявление статического поля  
public:  
    Student(const string& n) : name(n) {  
        totalCount++; // увеличиваем счётчик при создании  
    }  
  
    ~Student() {  
        totalCount--; // уменьшаем при уничтожении  
    }  
  
    static int getTotalCount() { // статический метод  
        return totalCount;  
    }  
}
```

```
};

// Определение статического поля (обязательно вне класса!)
int Student::totalCount = 0;

int main() {
    cout << "Студентов: " << Student::getTotalCount() << endl; // 0

    Student s1("Иван");
    Student s2("Мария");

    cout << "Студентов: " << Student::getTotalCount() << endl; // 2

    {
        Student s3("Пётр");
        cout << "Студентов: " << Student::getTotalCount() << endl; // 3
    } // s3 уничтожается

    cout << "Студентов: " << Student::getTotalCount() << endl; // 2
}
```

6. Паттерн «Одиночка» (Singleton)

Одиночка - порождающий паттерн проектирования, который гарантирует, что у класса есть только один экземпляр.

```
class Singleton {
private:
    static Singleton* instance;

    // Приватный конструктор - нельзя создать объект извне
    Singleton() {}

public:
    // Запрещаем копирование
    Singleton(const Singleton&) = delete;
    Singleton& operator=(const Singleton&) = delete;

    static Singleton* getInstance() {
        if (instance == nullptr) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

```

    }

    void doSomething() {
        cout << "Одиночка выполняет действие" << endl
    }
};

// Определение статического поля
Singleton* Singleton::instance = nullptr;

int main() {
    // Singleton s;          // ОШИБКА! конструктор приватный
    Singleton* s1 = Singleton::getInstance();
    Singleton* s2 = Singleton::getInstance();

    cout << "s1 == s2: " << (s1 == s2) << endl; // true - один и тот же объект
}

```

Примеры выполнения практической части

Пример 1. Композиция: классы «Дом» и «Комната»

Задача: Реализовать классы Room и House. Комнаты не могут существовать без дома (композиция).

```

#include <iostream>
#include <string>
#include <vector>

using namespace std;

class Room {
private:
    string name;
    double area;

public:
    Room(const string& n, double a) : name(n), area(a) {
        cout << "Создана комната \"" << name << "\" площадью " << area <<
        " м²" << endl;
    }

    ~Room() {
        cout << "Удалена комната \"" << name << "\" " << endl;
    }
}

```

```

    }

    void print() const {
        cout << " - " << name << ": " << area << " м²" << endl;
    }
};

class House {
private:
    string address;
    vector<Room> rooms; // композиция: комнаты внутри дома

public:
    House(const string& addr) : address(addr) {
        cout << "Построен дом по адресу: " << address << endl;
    }

    ~House() {
        cout << "Снесён дом по адресу: " << address << endl;
    }

    void addRoom(const string& name, double area) {
        rooms.emplace_back(name, area); // комната создаётся внутри дома
    }

    void print() const {
        cout << "Дом по адресу " << address << ":" << endl;
        for (const auto& room : rooms) {
            room.print();
        }
    }
};

int main() {
    setlocale(LC_ALL, "ru");

    House myHouse("ул. Цветочная, д. 15");

    myHouse.addRoom("Гостиная", 25.5);
    myHouse.addRoom("Кухня", 12.0);
    myHouse.addRoom("Спальня", 18.0);
}

```

```
myHouse print();
```

```
// При выходе из main дом уничтожается, а вместе с ним - все комнаты  
return 0;
```

```
}
```

Ожидаемый вывод:

Построен дом по адресу: ул. Цветочная, д. 15
Создана комната "Гостиная" площадью 25.5 м²
Создана комната "Кухня" площадью 12 м²
Создана комната "Спальня" площадью 18 м²
Дом по адресу ул. Цветочная, д. 15:
- Гостиная: 25.5 м²
- Кухня: 12 м²
- Спальня: 18 м²
Снесён дом по адресу: ул. Цветочная, д. 15
Удалена комната "Спальня"
Удалена комната "Кухня"
Удалена комната "Гостиная"

Пример 2. Агрегация: классы «Библиотека» и «Книга»

Задача: Реализовать классы Book и Library. Книги могут существовать без библиотеки (агрегация).

```
#include <iostream>
```

```
#include <string>
```

```
#include <vector>
```

```
using namespace std
```

```
class Book {
```

```
private:
```

```
    string title;
```

```
    string author;
```

```
public:
```

```
    Book(const string& t, const string& a) : title(t), author(a) {  
        cout << "Создана книга \"" << title << "\" (" << author << ")" << endl;  
    }
```

```
    ~Book() {
```

```
        cout << "Удалена книга \"" << title << "\" << endl;
```

```

    }

    void print() const {
        cout << " - \'" << title << "\", " << author << endl;
    }
};

class Library {
private:
    string name;
    vector<Book*> books; // агрегация: книги существуют независимо

public:
    Library(const string& n) : name(n) {
        cout << "Открыта библиотека \'" << name << "\'" << endl;
    }

    ~Library() {
        cout << "Закрыта библиотека \'" << name << "\'" << endl;
        // КНИГИ НЕ УНИЧТОЖАЮТСЯ!
    }

    void addBook(Book* book) {
        books.push_back(book);
        cout << "Книга \'" << book->getTitle() << "\"" добавлена в
библиотеку" << endl;
    }

    void print() const {
        cout << "Библиотека \'" << name << "\"" содержит:" << endl;
        for (const auto* book : books) {
            book->print();
        }
    }
};

// Нужен геттер для title (добавим в класс Book)
// В реальном коде нужно добавить метод getTitle()

int main() {
    setlocale(LC_ALL, "ru");

```

```

// Книги создаются независимо от библиотеки
Book b1("Война и мир", "Л. Толстой");
Book b2("Преступление и наказание", "Ф. Достоевский");
Book b3("Евгений Онегин", "А. Пушкин");

{
    Library lib("Городская библиотека");
    lib addBook(&b1);
    lib addBook(&b2);
    lib print();
} // Библиотека закрывается, но книги остаются

cout << "\nКниги всё ещё существуют после закрытия библиотеки:" <<
endl

b1.print();
b2.print();

return 0;
} // Книги удаляются только здесь

```

Пример 3. Статические члены: счётчик объектов

Задача: Реализовать класс Employee со статическим полем для подсчёта количества созданных объектов.

```

#include <iostream>
#include <string>

using namespace std;

class Employee {
private:
    string name;
    double salary;
    static int employeeCount; // статическое поле - счётчик сотрудников
    static double totalSalary; // статическое поле - сумма всех зарплат

public:
    Employee(const string& n, double s) : name(n), salary(s) {
        employeeCount++;
        totalSalary += salary;
        cout << "Создан сотрудник: " << name << ", зарплата: " << salary <<
endl

```

```

    }

~Employee() {
    employeeCount--;
    totalSalary -= salary;
    cout << "Уволен сотрудник: " << name << endl;
}

// Статические методы
static int getCount() {
    return employeeCount;
}

static double getAverageSalary() {
    if (employeeCount == 0) return 0;
    return totalSalary / employeeCount;
}

// Обычные методы
void setSalary(double newSalary) {
    totalSalary = totalSalary - salary + newSalary;
    salary = newSalary;
}

void print() const {
    cout << " " << name << ": " << salary << " руб." << endl;
}
};

// Определение статических полей
int Employee::employeeCount = 0;
double Employee::totalSalary = 0;

int main() {
    setlocale(LC_ALL, "ru");

    cout << "Всего сотрудников: " << Employee::getCount() << endl;
    cout << "Средняя зарплата: " << Employee::getAverageSalary() << endl;

    Employee e1("Иванов Иван", 50000);
    Employee e2("Петров Пётр", 60000);
    Employee e3("Сидорова Мария", 55000);

```

```

cout << "\nВсего сотрудников: " << Employee::getCount() << endl;
cout << "Средняя зарплата: " << Employee::getAverageSalary() << endl;

cout << "\nСписок сотрудников:" << endl;
e1.print();
e2.print();
e3.print();

e2.setSalary(65000);
cout << "\nПосле повышения зарплаты Петрова:" << endl;
cout << "Средняя зарплата: " << Employee::getAverageSalary() << endl;

return 0;
}

```

Индивидуальные задания

Базовый уровень (15 вариантов)

Общее задание: Реализовать два класса, связанных отношением композиции (часть не может существовать без целого). Продемонстрировать создание и уничтожение объектов.

№ вар.	Класс «Целое»	Класс «Часть»	Поля класса «Часть»
1	House	Room	название, площадь
2	Car	Wheel	диаметр, тип (зимняя/летняя)
3	Computer	CPU	модель, частота, ядра
4	University	Faculty	название, количество кафедр
5	Library	Bookshelf	номер, вместимость
6	Book	Chapter	название, количество страниц
7	School	Classroom	номер, вместимость
8	Apartment	Window	ширина, высота, материал
9	Phone	Battery	ёмкость (мАч), тип

№ вар.	Класс «Целое»	Класс «Часть»	Поля класса «Часть»
10	Ship	Deck	название, уровень
11	Building	Floor	номер, высота
12	Document	Page	номер, содержание (string)
13	Organization	Department	название, количество сотрудников
14	Train	Carriage	номер, тип (купе/плацкарт), мест
15	Guitar	String	номер, толщина

Средний уровень (15 вариантов)

Общее задание: Реализовать два класса, связанных отношением агрегация (часть может существовать без целого). Добавить статическое поле для подсчёта количества объектов класса «Часть». Продемонстрировать, что при уничтожении целого части продолжают существовать.

№ вар.	Класс «Целое»	Класс «Часть»	Поля класса «Часть»
1	Library	Book	название, автор, год
2	Team	Player	имя, номер, амплуа
3	Course	Student	имя, фамилия, группа
4	Playlist	Song	название, исполнитель, длительность
5	Group	Member	имя, роль
6	Fleet	Vehicle	модель, год выпуска, номер
7	Catalog	Product	название, цена, артикул

№ вар.	Класс «Целое»	Класс «Часть»	Поля класса «Часть»
8	Department	Employee	имя, должность, оклад
9	Menu	Dish	название, цена, калории
10	Squad	Soldier	звание, имя, опыт
11	Roster	Athlete	имя, вид спорта, рекорд
12	Schedule	Lesson	предмет, время, аудитория
13	Album	Photo	название, дата, размер
14	Cart	Item	название, цена, количество
15	Wardrobe	Clothing	тип, размер, цвет

Продвинутый уровень (15 вариантов)

Общее задание: Реализовать один из вариантов ниже. В каждом варианте необходимо:

Реализовать отношение композиция или агрегация согласно варианту

Добавить статическое поле для подсчёта объектов одного из классов

Реализовать паттерн «Одиночка» (Singleton) для одного из классов

Продемонстрировать работу всех реализованных возможностей

№ вар.	Задание
1	Система учёта студентов: University (одиночка) + Student (агрегация). Подсчёт количества студентов
2	Система управления библиотекой: Library (одиночка) + Book (агрегация). Подсчёт количества книг
3	Система учёта сотрудников: Company (одиночка) + Employee (агрегация). Подсчёт сотрудников и средней зарплаты
4	Система заказов в ресторане: Restaurant (одиночка) + Order (агрегация) + Dish (композиция в Order)

№ вар.	Задание
5	Система управления курсами: TrainingCenter (одиночка) + Course (агрегация) + Student (агрегация в Course)
6	Система управления таксопарком: TaxiCompany (одиночка) + Car (агрегация) + Driver (агрегация)
7	Система учета поставок: Warehouse (одиночка) + Product (агрегация) + Supplier (агрегация)
8	Система управления гостиницей: Hotel (одиночка) + Room (агрегация) + Booking (композиция в Room)
9	Система управления проектами: ProjectManager (одиночка) + Project (агрегация) + Task (композиция в Project)
10	Система интернет-магазина: Store (одиночка) + Customer (агрегация) + Purchase (композиция)
11	Система управления автопарком: Depot (одиночка) + Bus (агрегация) + Route (агрегация в Bus)
12	Система управления кинотеатром: Cinema (одиночка) + Movie (агрегация) + Session (композиция в Movie)
13	Система управления больницей: Hospital (одиночка) + Doctor (агрегация) + Patient (агрегация) + Appointment (композиция)
14	Система управления спортивным клубом: SportsClub (одиночка) + Coach (агрегация) + Section (агрегация) + Member (агрегация в Section)
15	Система управления банком: Bank (одиночка) + Client (агрегация) + Account (композиция в Client)

Контрольные вопросы

1. Что такое композиция? Приведите пример.
2. Что такое агрегация? Чем она отличается от композиции?
3. Как реализовать композицию в C++? Приведите пример кода.

4. Как реализовать агрегацию в C++? Приведите пример кода.
5. В чём разница между временем жизни частей при композиции и агрегации?
6. Что такое статическое поле класса? Чем оно отличается от обычного поля?
7. Как объявить и определить статическое поле? Где оно хранится?
8. Что такое статический метод? Можно ли из него обратиться к нестатическим полям?
9. Для чего используются статические счётчики объектов?
10. Что такое паттерн «Одиночка» (Singleton)? Для чего он нужен?
11. Как запретить создание копий объекта (конструктор копирования)?
12. В каком порядке вызываются конструкторы и деструкторы при композиции?
13. Можно ли изменить статическое поле из обычного метода?
14. Почему статические поля нужно определять вне класса?
15. В каких случаях стоит использовать композицию, а в каких - агрегацию?

Требования к отчёту

Титульный лист с названием работы, ФИО студента, номером группы, вариантом

Цель работы (формулируется самостоятельно на основе предложенной)

Теоретическая часть (основные понятия: композиция, агрегация, статические члены)

Постановка задачи (текст задания согласно варианту)

Листинг программы с комментариями

Результаты работы (скриншоты вывода программы)

Ответы на контрольные вопросы (письменно, развёрнуто)

Выводы (что нового узнали, какие навыки приобрели)

Ссылка на Git-репозиторий с кодом программы

Критерии оценивания

Базовый уровень (максимум 60 баллов)

Критерий	Баллы
Реализованы два класса с отношением композиция	20
Демонстрация создания и уничтожения объектов	15
Корректное управление памятью (нет утечек)	15

Критерий	Баллы
Код отформатирован, есть комментарии	10

Средний уровень (максимум 80 баллов)

Критерий	Баллы
Все требования базового уровня	40
Реализована агрегация (два класса)	15
Реализовано статическое поле для подсчёта объектов	15
Демонстрация, что при уничтожении целого части продолжают существовать	10

Продвинутый уровень (максимум 100 баллов)

Критерий	Баллы
Все требования среднего уровня	60
Реализация паттерна «Одиночка» (Singleton)	15
Реализация статических методов для работы со статическими полями	10
Корректная обработка всех граничных случаев	10
Наличие ссылки на Git-репозиторий	5

Штрафные баллы

Нарушение	Штраф
Утечка памяти (при композиции)	-20
Неправильное использование статических полей	-10
Отсутствует проверка на nullptr в агрегации	-10
Отсутствуют комментарии в коде	-5

Нарушение	Штраф
Нет ссылки на Git-репозиторий	-10
Программа не компилируется	работа не принимается

Итоговая шкала

Оценка (традиционная)	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74
Неудовлетворительно	менее 60